

PSTAT 10 Data Science Principles

Lecture 12: Advanced Simulation

John Robin Inston 

johninston@ucsb.edu

UC Santa Barbara

August 20, 2025

Introduction

Review: Lecture 11

👉 Last lecture we had yet another **very math heavy** lecture covering:

- ▶ Poisson Distributions
- ▶ Generalizing R Functions
- ▶ Continuous Random Variables
- ▶ Continuous Uniform Distribution
- ▶ Normal (Gaussian) Distribution

Overview: Lecture 12

👉 This lecture we will conclude our study of probability by looking at:

- ▶ Simulation
- ▶ Harder probability questions
- ▶ Monte Carlo methods

Note: This will be our last R focused lecture until lecture 19 but R will still appear in homework and section exercises.

Simulating Pi

History of Estimating π

One of the oldest **approximation problems** humans have been working on is estimating π .

There is evidence of estimating π as far back as:

- ▶ Ancient Babylon and Egypt (1700 - 1600 BCE)
- ▶ Ancient India (600 BCE)
- ▶ Ancient Greece (300 BCE)
- ▶ Ancient China (250 CE)

More recently we have estimated π using:

- ▶ Infinite series and trigonometric identities (middle ages)
- ▶ Rapidly converging infinite series (19th & 20th centuries)
- ▶ **Supercumpoters** (modern day)

Estimating Pi

We attempt to **construct our own estimate** of π using simulation using the unit circle.

Area of a Circle

Recall from basic trigonometry that the area of a circle is given by:

$$A = \pi \times r^2.$$

When considering a unit circle $r = 1$ and so

$$A = \pi.$$

Hence if we can **approximate the area of a unit circle** we approximate π .

Approximate Unit Circle Area

To approximate the area of the unit circle our approach is as follows:

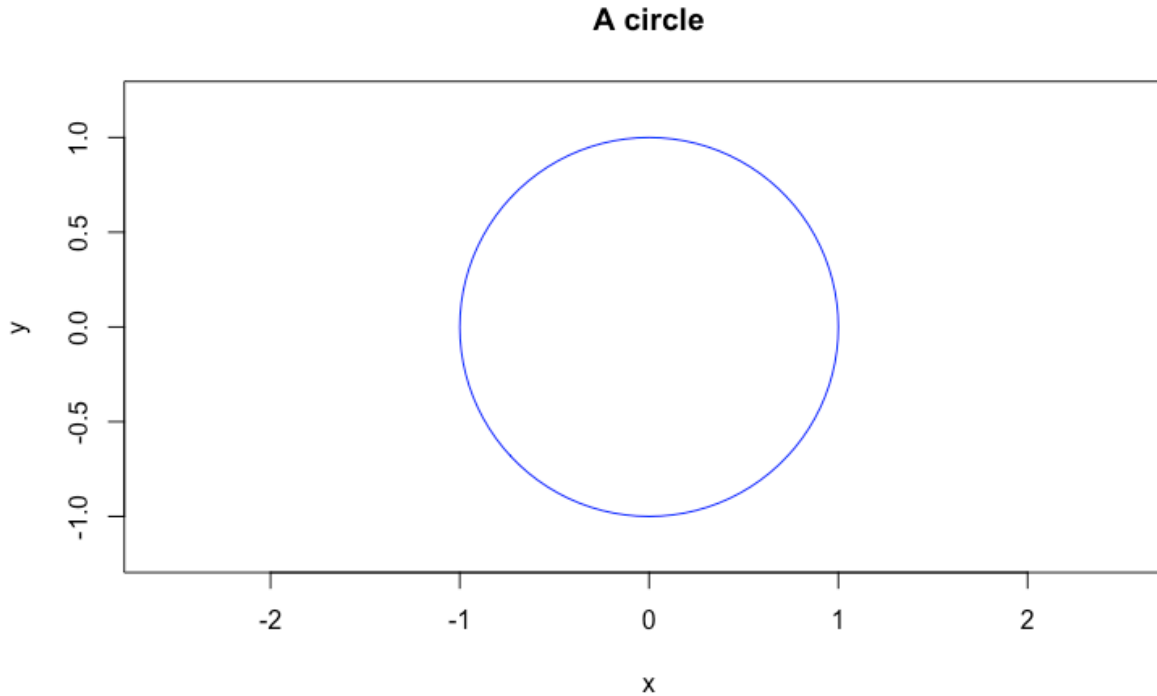
1. Draw a unit circle.
2. Enclose the unit circle perfectly in a unit square.
3. Simulate points uniformly in the square.
4. Determine the proportion of points that land within the unit circle.
5. This proportion is an approximation of the area of the circle π over the area of the square 4 and so we compute:

$$\pi \approx 4 \times \text{Prop}$$

Approximate Unit Circle Area

Draw a Circle

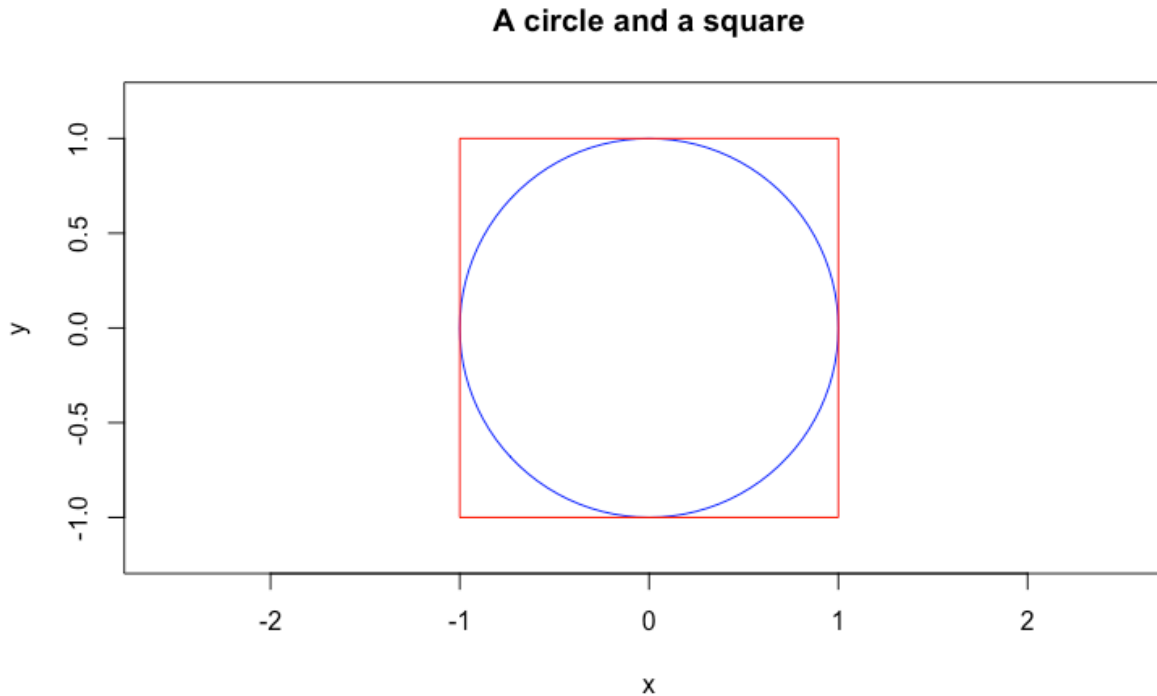
We draw a circle!



Approximate Unit Circle Area

Draw a Square

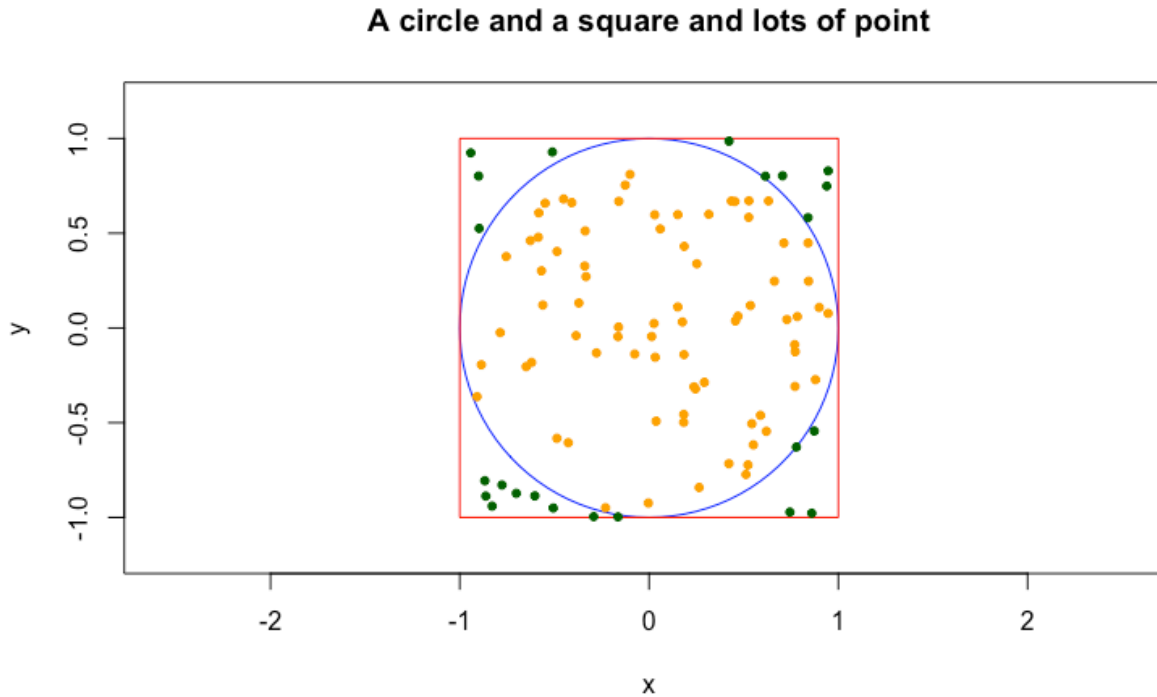
Then perfectly draw a square around our circle:



Approximate Unit Circle Area

Simulate Points

Then simulate a lot of points uniformly within the square:



Approximate Unit Circle Area

Compute the proportion

Here we have 77 points within the circle and so we compute

$$\pi \approx 4 \times 0.77 = 3.08.$$

⬆ To increase our accuracy we increase the number of simulations.

Approximate Unit Circle Area

Compute the proportion

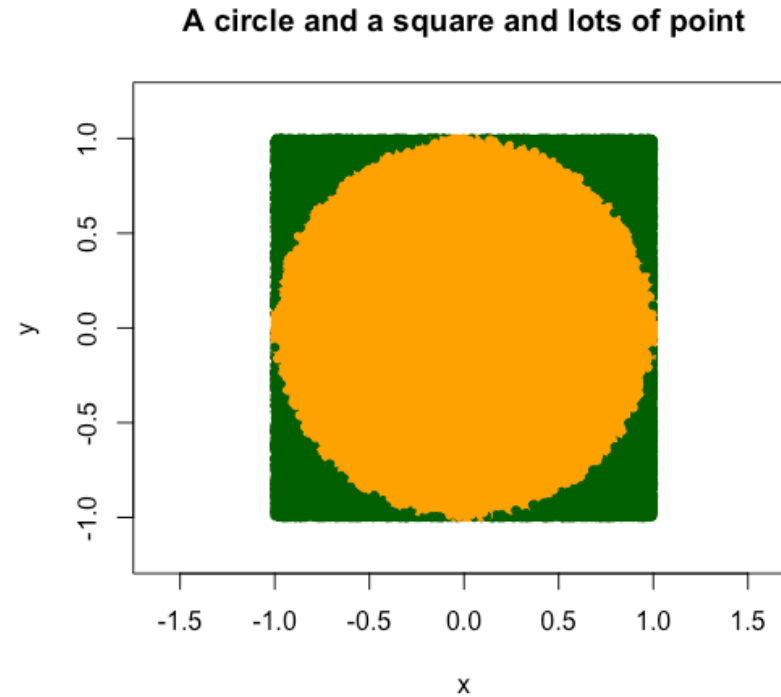
Here we have 77 points within the circle and so we compute

$$\pi \approx 4 \times 0.77 = 3.08.$$

 To increase our accuracy we increase the number of simulations.

Lets up our simulation count from 100 points to 100,000 points we get the following approximation.

Increasing Accuracy



Here our approximation is 3.13816.

Monte Carlo Methods

Monte Carlo Methods

Monte Carlo (MC) methods are a study of these types of problems:

- ▶ Consider a deterministic but hard to measure value (such as π).
- ▶ Assume we have access to **controlled random sampling** (e.g. `runif()`).
- ▶ We can approximate the deterministic value from a large number of samples.

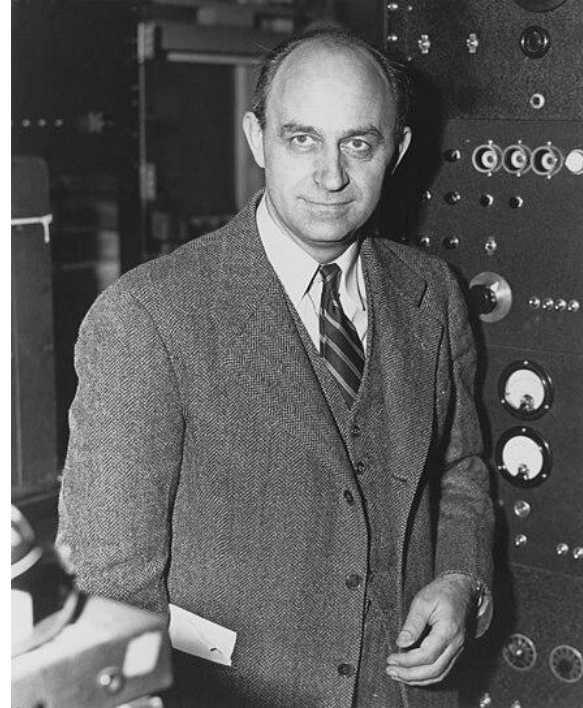


Figure 1: Enrico Fermi, an early MC practitioner.

MC Considerations - Computational Cost

Consideration 1: Time and Computation Effort.

```
t1 ← Sys.time()  
rands ← runif(100)  
t2 ← Sys.time()  
t2 - t1
```

Time difference of 0.001913786 secs

```
t1 ← Sys.time()  
rands ← runif(1000000)  
t2 ← Sys.time()  
t2 - t1
```

Time difference of 0.0553019 secs

MC Considerations - Error Bounds

Consideration 2: Error Bounds

```
# estimating pi using our previous method
m ← 100; x ← runif(m, -1, 1); y ← runif(m, -1, 1)
distances ← sqrt(x^2 + y^2)
est ← length(which(distances < 1)) * 4 / m
(error ← abs(est - pi))
```

```
[1] 0.01840735
```

```
m ← 100000; x ← runif(m, -1, 1); y ← runif(m, -1, 1)
distances ← sqrt(x^2 + y^2)
est ← length(which(distances < 1)) * 4 / m
(error ← abs(est - pi))
```

```
[1] 0.01100735
```

Evaluating MC Methods

These considerations are essential in evaluating MC methods.

- ▶ How **quickly** can we simulate trials?
- ▶ What **sort of error** do we have?
- ▶ How **fast does the approach converge**?

Ideal Scenario: Method that is easy to do (fast sampling), with predictable error rates that rapidly converges to the true value.

- ▶ This **usually doesn't exist**!
- ▶ Generally need to make a **trade off** between speed and accuracy.

Buffon's Needle

Buffon's Needle

Buffon's needle is a famous thought problem from the 1700s:

- ▶ Suppose we have a floor made out of wooden planks evenly spaced some distance d apart.
- ▶ If we drop a needle of length 1 onto the floor what is the probability that it crosses between two planks?

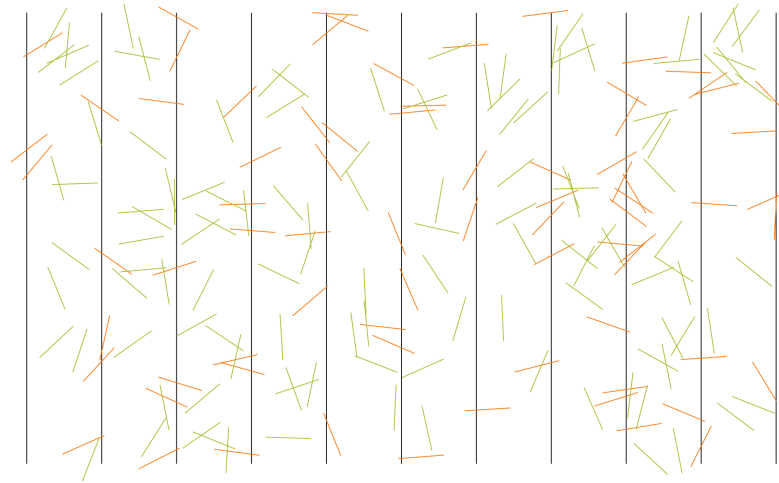


Figure 1: Buffon's Needle

Buffon's Needle

Interestingly, the probability p has a closed form expression which we can rearrange to estimate π :

$$p = \frac{2}{\pi} \times \frac{1}{d} \implies \pi = \frac{2}{p} \times \frac{1}{d}.$$

Buffon's Needle

Interestingly, the probability p has a closed form expression which we can rearrange to estimate π :

$$p = \frac{2}{\pi} \times \frac{1}{d} \implies \pi = \frac{2}{p} \times \frac{1}{d}.$$

We will **simulate the simplest case**:

- ▶ Board distance $d = \text{needle length} = 1$
- ▶ Needle angle $= \theta$
- ▶ Distance from needle center to nearest plank $= D$
- ▶ Needle intersect lines when $D > 0.5 \times \sin(\theta)$.

Buffon's Needle

Buffon's Needle Simplest Case

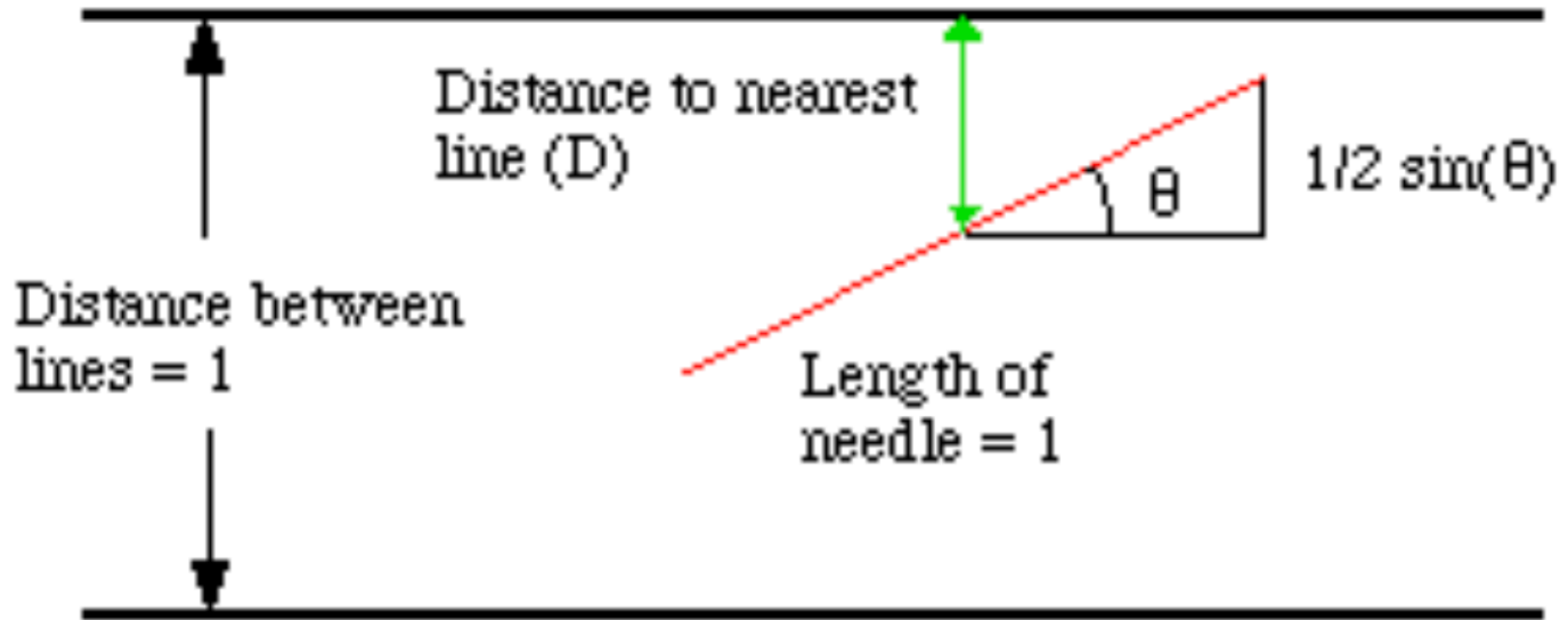


Figure 2: Buffon's Needle Diagram

Buffon's Needle Simulation

We conduct the following simulation:

```
m ← 1000  
  
D ← runif(m, 0, 0.5) # randomly place needle centers  
theta ← runif(m, 0, pi) # randomly select angles  
p ← sum(D < 0.5*sin(theta))/m # number with D < 0.5*sin(θ)
```

Buffon's Needle Simulation

We conduct the following simulation:

```
m ← 1000  
  
D ← runif(m, 0, 0.5) # randomly place needle centers  
theta ← runif(m, 0, pi) # randomly select angles  
p ← sum(D < 0.5*sin(theta))/m # number with D < 0.5*sin(θ)
```

We obtain the following estimate for π .

```
(pi_est ← 2/p)
```

```
[1] 3.095975
```

Buffon's Needle Simulation

We can increase our accuracy by increasing our simulation count.

```
m ← 1000000  
  
D ← runif(m, 0, 0.5) # randomly place needle centers  
theta ← runif(m, 0, pi) # randomly select angles  
p ← sum(D < 0.5*sin(theta))/m # number with D < 0.5*sin(θ)
```

Buffon's Needle Simulation

We can **increase our accuracy** by increasing our simulation count.

```
m ← 1000000  
  
D ← runif(m, 0, 0.5) # randomly place needle centers  
theta ← runif(m, 0, pi) # randomly select angles  
p ← sum(D < 0.5*sin(theta))/m # number with D < 0.5*sin(θ)
```

We obtain the following new estimate for π .

```
(pi_est ← 2/p)
```

```
[1] 3.144852
```

Euler's Number

Euler's Number

Another interesting mathematical constant is Euler's number e .

```
exp(1)
```

```
[1] 2.718282
```

We have worked with e every time we have used the `exp()` function which is defined as:

$$\exp(x) = e^x$$

i.e. raising e to the power of the input of the function.

- ▶ It is therefore the **base of the natural logarithm** $\ln$.
- ▶ It also appears in a **large number** of the standard distribution functions (e.g. Poisson, Exponential, Normal, Gamma).

Exercise - Lex Fridman Problem

Consider how we might estimate e with Monte Carlo simulation?

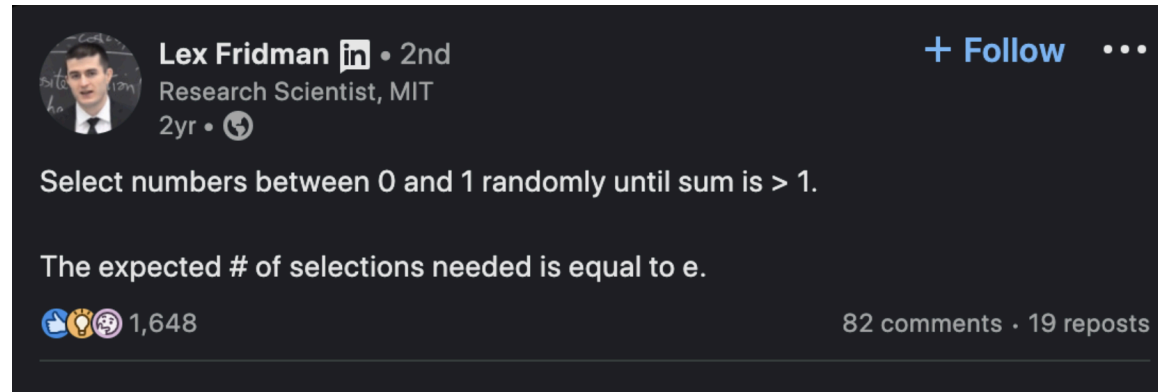


Figure 3: Lex Fridman Problem

Write and replicate a function that:

1. Select numbers with `runif()` until the sum is > 1 ;
2. Determine how many selections it takes; and
3. Repeat $m=10000$ times to approximate e .

Solution - Lex Fridman Problem

```
above_one <- function() {  
  i <- 0  
  count <- 0  
  
  while (i < 1) {  
    i <- i + runif(1, 0, 1)  
    count <- count + 1  
  }  
  return(count)  
}  
  
m <- 10000  
mean(replicate(m, above_one()))
```

```
[1] 2.7171
```


Golden Ratio

Golden Ratio

The **golden ratio** ϕ is another important mathematical constant which is defined as

$$\phi = \frac{a}{b} \quad s.t. \quad \frac{a+b}{a} = \frac{a}{b}.$$

We have encountered ϕ previously when working with the **Fibonacci sequence**.

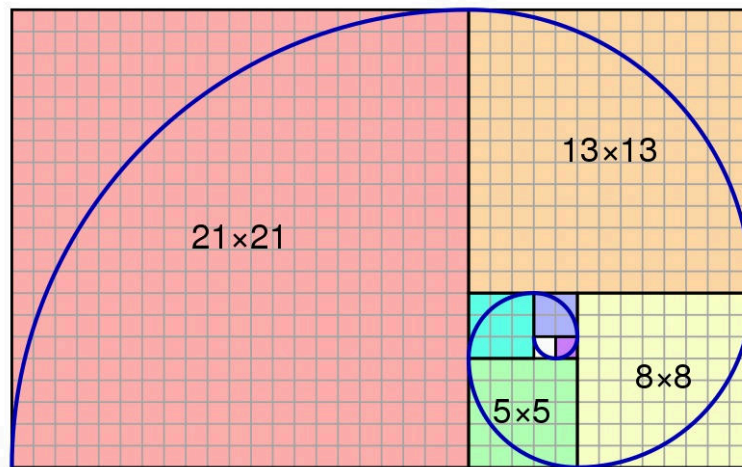


Figure 4: Golden Ratio

Golden Ratio MC Estimation

We estimate the golden ratio as follows:

1. Generate pairs of numbers a, b such that $a > b$ at random;
2. Compute $U = (a + b)/a$ and $L = a/b$ for each pair;
3. Compute the ratio of U/L ; and
4. Return the a, b values giving U/L closest to 1.

This approach requires many trials.

Golden Ratio Approximation

```
golden ← function() {  
  a ← runif(1, 0, 1)  
  b ← runif(1, 0, a)  
  U ← (a + b) / a  
  L ← a / b  
  ratio ← U / L  
  return(c(ratio, a, b))  
}  
vals ← replicate(10000, golden())  
estimates ← which.min(abs(vals[,1] - 1))  
(phi ← vals[,estimates][2]/vals[,estimates][3])
```

```
[1] 1.618178
```

Recall that the actual value is: 1.61803...

Monte Carlo Integration

Monte Carlo Integration

We compute the following integral via simulation

$$\int_0^5 x^3 dx.$$

Granted this is not a very interesting integral but at least it is quite simple.

Monte Carlo Integration

We compute the following **integral via simulation**

$$\int_0^5 x^3 dx.$$

Granted this is not a very interesting integral but at least it is quite simple.

Idea: Express the integral as the **expectation of a random variable**, say $X \sim \mathcal{U}(0, 5)$ with PDF

$$f_X(x) = \frac{1}{5} \quad \text{for } 0 < x < 5.$$

Rewrite the Integral

We can rewrite the integral as follows:

$$\begin{aligned}\int_0^5 x^3 dx &= \int_0^5 5x^3 \times 1/5 dx \\ &= \int_0^5 5x^3 f_X(x) dx \\ &= \mathbb{E}[5X^3].\end{aligned}$$

Where $X \sim \mathcal{U}(0, 5)$. The problem reduces to:

- ▶ Simulating many $\mathcal{U}(0, 5)$ random variables;
- ▶ Cubing all simulations and multiplying them by 5
- ▶ Taking the mean to estimate the integral.

Approximate the Integral

```
set.seed(24)
m ← 10000
sims ← runif(m, 0, 5)
mean(5 * sims^3)
```

```
[1] 154.0979
```

Computing the **actual value analytically** we have:

$$\int_0^5 x^3 dx = \left[\frac{x^4}{4} \right]_0^5 = \frac{5^4}{4} = 156.25.$$

The Monty Hall Problem

The Monty Hall Problem

The **Monty Hall problem** is another famous probability puzzle with the following setup:

- ▶ You are a contestant on a game show;
- ▶ You are shown **three doors**: a **new car** is behind one door and a **goat** is behind each of the others;
- ▶ You **select a door at random** and have the chance to keep whatever is behind the door;
- ▶ After choosing a door, the show host opens one of the two remaining doors to show a goat at which point they ask you whether you would like to **change your guess**.

The Monty Hall Problem

The **Monty Hall problem** is another famous probability puzzle with the following setup:

- ▶ You are a contestant on a game show;
- ▶ You are shown **three doors**: a **new car** is behind one door and a **goat** is behind each of the others;
- ▶ You **select a door at random** and have the chance to keep whatever is behind the door;
- ▶ After choosing a door, the show host opens one of the two remaining doors to show a goat at which point they ask you whether you would like to **change your guess**.

Question: Should you change your guess?

The Monty Hall Solution

Solution: It is **always** better to switch doors!

The Monty Hall Solution

Solution: It is **always** better to switch doors!

Computing the odds of winning using **conditional probability** we find that:

- ▶ your odds of winning if you choose to stay are $1/3$ whilst;
- ▶ your odds of winning if you choose to change are $2/3$.

This seems **counter intuitive**! Surely it is just 50/50 i.e. $1/2$ for both, there are two doors and 1 is the winner?

Monty Hall Simulation

```
monty_hall_game ← function(strategy) {  
  prize_door ← sample(1:3, 1) # set door with car  
  player_choice ← sample(1:3, 1) # set player choice  
  remaining_doors ← setdiff(1:3, c(prize_door, player_choice))  
  if (length(remaining_doors) == 1) { revealed_door ← remaining_doors}  
  else { revealed_door ← sample(remaining_doors, 1)} # open door  
  if (strategy == "stay") { # stay put  
    final_choice ← player_choice  
  } else { # switch doors  
    options ← setdiff(1:3, c(revealed_door, player_choice))  
    if (length(options) == 1) {  
      final_choice ← options  
    } else {  
      final_choice ← sample(options, 1)  
    }  
  }  
  win ← final_choice == prize_door; return(win)  
}
```

Monty Hall Simulation Breakdown

We initially **set the door with prize**, selected door and remaining door.

```
monty_hall_game ← function(strategy) {  
  prize_door ← sample(1:3, 1)  
  player_choice ← sample(1:3, 1)  
  remaining_doors ← setdiff(1:3, c(prize_door, player_choice))  
}
```

We then **simulate the host picking a door**.

```
if (length(remaining_doors) == 1) {  
  revealed_door ← remaining_doors  
} else {  
  revealed_door ← sample(remaining_doors, 1) # open door  
}
```


Monty Hall Simulation Breakdown

We then **decide whether to switch of stick** with current door.

```
if (strategy == "stay") {  
  final_choice ← player_choice  
} else {  
  options ← setdiff(1:3, c(revealed_door, player_choice))  
  if (length(options) == 1) {  
    final_choice ← options  
  } else {  
    final_choice ← sample(options, 1)  
  }  
}
```

Before finally reporting the results.

```
win ← final_choice == prize_door  
  
return(win)
```

Testing Claim

```
set.seed(24)
m ← 10000
mean(replicate(m, monty_hall_game("stay")))
```

```
[1] 0.3384
```

```
mean(replicate(m, monty_hall_game("switch")))
```

```
[1] 0.6578
```

Summary

Topics Covered

We have now looked through several examples of more complex simulation including simulating:

- ▶ Pi;
- ▶ Buffon's Needle;
- ▶ Euler's Number;
- ▶ Golden Ratio;
- ▶ Integrals; and
- ▶ Monty Hall Problem

Thats all we need for probability for PSTAT10!

Next Class

Next class we will be moving away from RStudio and will instead be focusing on:

- ▶ Databases; and
- ▶ Introduction to SQL.